10 Boolean logic

In this chapter, you will learn about:

- ★ the identification, definition, symbols and functions of the standard logic gates: NOT, AND, OR, NAND, NOR and XOR
- ★ how to use logic gates to create logic circuits from:
  - a given problem
  - a logic expression
  - a truth table
- ★ how to complete truth tables from:
  - a given problem
  - a logic expression
  - a logic circuit
- ★ how to write a logic expression from:
  - a given problem
  - a logic circuit
  - a truth table.

10.1 Standard logic gate symbols

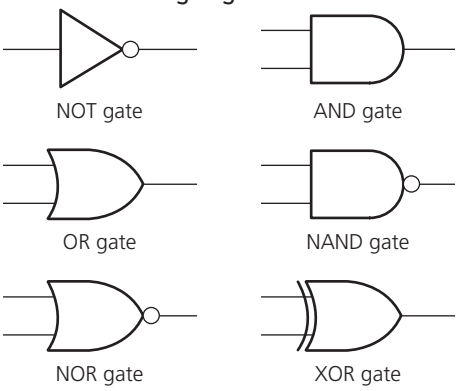
Electronic circuits in computers, solid state drives and controlling devices are made up of thousands of **logic gates**. Logic gates take binary inputs and produce a binary output. Several logic gates combined together form a **logic circuit** and these circuits are designed to carry out a specific function.

The checking of the output from a logic gate or logic circuit is done using a **truth table**.

This chapter will consider the function and role of logic gates, logic circuits and truth tables. Also a number of possible applications of logic circuits will be considered. A reference to **Boolean algebra** will be made throughout the chapter; but this is really outside the scope of this text book. However, Boolean algebra will be seen on many logic gate websites and is included here for completeness, since many students may prefer this notation to logic statements.

10.1.1 Logic gate symbols

Six different logic gates will be considered in this chapter:



▲ **Figure 10.1** Logic gate symbols

Truth tables

Truth tables are used to trace the output from a logic gate or logic circuit. The NOT gate is the only logic gate with one input; the other five gates have two inputs (see Figure 10.1).

Although each logic gate can only have one or two inputs, the number of inputs to a logic circuit can be more than 2; for example, three inputs give a possible 2^3 (=8) binary combinations. And for four inputs, the number of possible binary combinations is 2^4 (=16). It is clear that the number of possible binary combinations is a multiple of the number 2 in every case. The possible inputs in a truth table can be summarised as shown in Table 10.1.

▼ **Table 10.1** All possible inputs for truth tables with two, three and four inputs

Inputs	
A	B
0	0
0	1
1	0
1	1

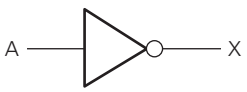
Inputs		
A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Inputs			
A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

As we can see, a truth table will also list the output for every possible combination of inputs.

10.2 The function of the six logic gates

10.2.1 NOT gate



▲ **Figure 10.2**

Description:	Truth table:	How to write this:
The output, X, is 1 if: the input, A, is 0	▼ **Table 10.2** **Input: A** / **Output: X** 0 / 1 1 / 0	X = NOT A (logic notation) X = $\overline{A}$ (Boolean algebra)

Note the use of Boolean algebra to represent logic gates. This is optional at IGCSE but many students may prefer to use this notation (see NOTE later).

10.2.2 AND gate



▲ **Figure 10.3**

Description:	Truth table:	How to write this:
The output, X, is 1 if: both inputs, A and B, are 1	▼ **Table 10.3** **Inputs: A, B** / **Outputs: X** 0 0 0 0 1 0 1 0 0 1 1 1	X = A AND B (logic notation) X = A . B (Boolean algebra)

10.2.3 OR gate



▲ **Figure 10.4**

Description:	Truth table:	How to write this:
The output, X, is 1 if: either input, A or B, or both, are 1	▼ **Table 10.4** **Inputs: A, B** / **Output: X** 0 0 0 0 1 1 1 0 1 1 1 1	X = A OR B (logic notation) X = A + B (Boolean algebra)

10.2.4 NAND gate (NOT AND)



▲ **Figure 10.5**

Description:	Truth table:	How to write this:
The output, X, is 1 if: input A AND input B are NOT both 1	▼ **Table 10.5** **Inputs** / **Output** **A** **B** **X** 0 0 1 0 1 1 1 0 1 1 1 0	X = A NAND B (logic notation) $X = \overline{A \cdot B}$ (Boolean algebra)

10.2.5 NOR gate (NOT OR)



▲ **Figure 10.6**

Description:	Truth table:	How to write this:
The output, X, is 1 if: neither input A nor input B is 1	▼ **Table 10.6** **Inputs** / **Output** **A** **B** **X** 0 0 1 0 1 0 1 0 0 1 1 0	X = A NOR B (logic notation) $X = \overline{A + B}$ (Boolean algebra)

10.2.6 XOR gate



▲ **Figure 10.7**

Description:	Truth table:	How to write this:
The output, X, is 1 if: (input A is 1 AND input B is 0) **or** (input A is 0 AND input B is 1)	▼ **Table 10.7** **Inputs** / **Output** **A** **B** **X** 0 0 0 0 1 1 1 0 1 1 1 0	X = A XOR B (logic notation) $X = (A \cdot \overline{B}) + (\overline{A} \cdot B)$ (Boolean algebra) NOTE: this is sometimes written as: $(A + B) \cdot \overline{(A \cdot B)}$

Activity 10.1

Show why X = (A AND NOT B) OR (NOT A AND B)
and
Y = (A OR B) AND (NOT (A AND B)) both represent the same logic gate.

You will notice in the Boolean algebra, three new symbols; these have the following meaning:

- . represents the AND operation
- + represents the OR operation
- a bar (above the letter or letters, e.g. $\overline{a}$) represents the NOT operation.

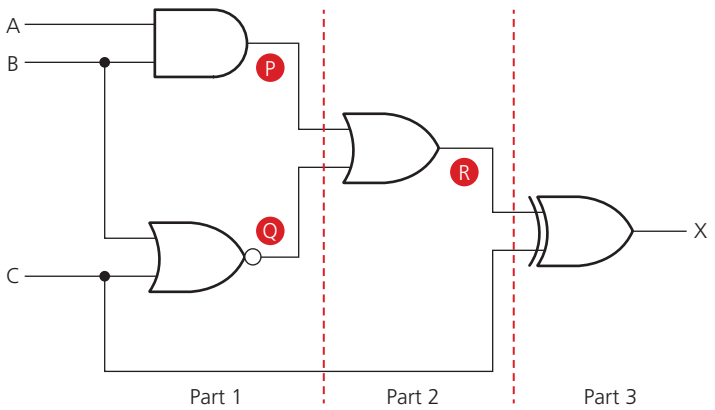
10.3 Logic circuits, logic expressions, truth tables and problem statements

When logic gates are combined together to carry out a particular function, such as controlling a robot, they form a logic circuit. The following eight examples show how to carry out the following tasks:

- Create a logic circuit from a:
  - problem statement (examples 6 and 7)
  - logic or Boolean expression (examples 3 and 8)
  - truth table (examples 4 and 5)
- Complete a truth table from a:
  - problem statement (examples 6 and 7)
  - logic or Boolean expression (examples 3 and 8)
  - logic circuit (example 1)
- Write a logic or Boolean expression from a:
  - problem statement (examples 6 and 7)
  - logic circuit (example 2)
  - truth table (examples 4 and 5).

Example 1

Produce a truth table for the following logic circuit (note the use of black circles at the junctions between wires):



▲ **Figure 10.8**

There are three inputs to this logic circuit, therefore, there will be eight possible binary values that can be input.

To show stepwise how the truth table is produced, the logic circuit has been split up into three parts, as shown by the dotted lines, and intermediate values are shown as P, Q and R.